

Programação de Sistemas Linux

Aula 03 — Ferramentas de Programação (Parte 1)

Hugo Marcondes

Departamento Acadêmico de Eletrônica
DAELN

hugo.marcondes@ifsc.edu.br



Abertura

Notes

Notes

Objetivos do encontro

Ao final deste encontro, você será capaz de:

- ✓ Criar um repositório **Git** e praticar o fluxo básico: add, commit, branch, merge
- ✓ Usar o **Valgrind** para encontrar vazamentos de memória (*memory leaks*)
- ✓ Rastrear as chamadas de sistema de um programa com **strace**
- ✓ Descrever um projeto C com múltiplos arquivos usando **CMake**

Regra de hoje

Quatro ferramentas, uma tacada só. Cada uma resolve um problema que o GCC/Make/GDB sozinhos não resolvem.



Notes

Git: controle de versão

Notes

O problema

```
trabalho_final.c, trabalho_final_v2.c,  
trabalho_final_v2_CORRIGIDO.c,  
trabalho_final_v2_CORRIGIDO_final.c...familiar?
```

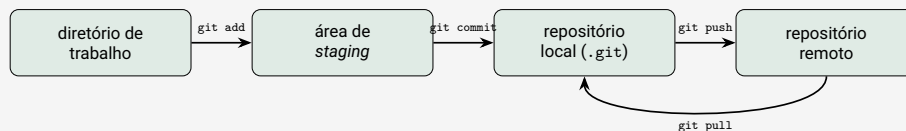
Git resolve isso:

- Guarda o **histórico completo** de mudanças do projeto
- Permite **voltar** a qualquer versão anterior
- Permite trabalhar em **paralelo** sem sobrescrever o trabalho de outra pessoa
- Base de praticamente todo fluxo de trabalho profissional em software



Notes

Do arquivo ao repositório remoto



- **Staging (index):** o que vai entrar no próximo commit
- **Commit:** uma **fotografia** do projeto, com mensagem e autor
- O repositório remoto (ex.: GitHub) é **opcional** – Git funciona 100% localmente



Notes

Primeiros passos

```
1 $ git init                # cria o repositório (.git/)
2 $ git status              # o que mudou desde o último commit?
3 $ git add main.c estatistica.c # marca arquivos para o próximo commit
4 $ git commit -m "Implementa cálculo de estatísticas"
5 $ git log --oneline       # histórico resumido de commits
6
```

- `git status` é o comando que você vai digitar **o tempo todo**
- Mensagens de commit **curtas e descritivas** — no imperativo: “Adiciona”, “Corrige”, “Remove”
- `git add .` adiciona **tudo** que mudou — use com cuidado

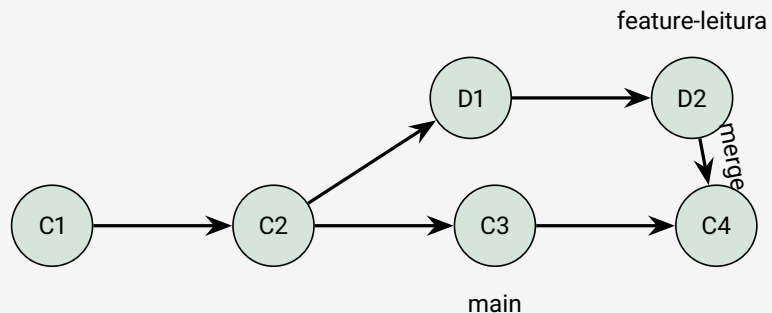


Notes

Branches: trabalhando em paralelo

Uma *branch* é uma linha independente de desenvolvimento.

```
1 $ git branch              # lista as branches existentes
2 $ git switch -c feature-leitura # cria e muda para uma nova branch
3 $ git switch main         # volta para a branch principal
4 $ git branch -d feature-leitura # apaga uma branch (já mesclada)
5
```



Notes

Merge e conflitos

```
1 $ git switch main
2 $ git merge feature-leitura
3
```

Quando as duas branches mudaram a **mesma linha**, o Git não decide sozinho:

```
1 <<<<<< HEAD
2     int tamanho = ler_tamanho_teclado();
3     =====
4     int tamanho = ler_tamanho_arquivo("dados.txt");
5 >>>>>> feature-leitura
6
```

Resolvendo o conflito

Edite o arquivo, decida qual código fica (ou combine os dois), remova as marcações <<<<<<, =====, >>>>>>, depois `git add + git commit` para concluir o merge.



Notes

.gitignore e boas práticas

Definição

.gitignore lista arquivos e pastas que o Git deve **ignorar** – geralmente arquivos gerados pelo build, que não fazem sentido versionar.

```
1 # .gitignore de um projeto C
2 *.o
3 *.out
4 build/
5 *.exe
6
```

- Versione **código-fonte**, não **binários gerados**
- Um commit por **mudança lógica** – não um commit gigante no fim do dia
- `git diff` antes de `git add` para revisar o que vai entrar



Notes

Valgrind: detecção de memory leaks

Notes

O problema: memória que ninguém devolve

Definição

Memory leak: bloco alocado com `malloc()` / `calloc()` / `realloc()` que o programa **nunca libera** com `free()`, mesmo sem precisar mais dele.

- Em C, **não existe coletor de lixo** — você gerencia a memória manualmente
- Um leak isolado não trava nada **na hora** — mas em programas de longa duração (servidores, daemons), leaks se acumulam até esgotar a RAM
- Detectar “a olho” é praticamente impossível em código real

Notes



```
1 $ gcc -g -Wall -Wextra -std=c11 -o programa programa.c
2 $ valgrind --leak-check=full ./programa
3
```

- Compile sempre com `-g` — o Valgrind precisa dos símbolos de depuração para apontar a **linha exata** do `malloc()` culpado
- `-leak-check=full` detalha **cada** bloco perdido, não só um resumo
- O programa roda **normalmente**, só (bem) mais devagar — o Valgrind simula uma CPU para observar cada acesso à memória



Exemplo real: um leak em produção

```
1 int *processa_lote(int tamanho) {
2     int *buffer = malloc(tamanho * sizeof(int)); LEAK!
3     for (int i = 0; i < tamanho; i++) {
4         buffer[i] = i * i;
5     }
6     return NULL; // buffer nunca e devolvido nem liberado
7 }
8
```

```
==12345== HEAP SUMMARY:
==12345==    in use at exit: 400 bytes in 1 blocks
==12345==   total heap usage: 3 allocs, 2 frees, 1,224 bytes allocated
==12345==
==12345== 400 bytes in 1 blocks are definitely lost in loss record 1 of 1
==12345==    at 0x4C3283B: malloc (vg_replace_malloc.c:299)
==12345==    by 0x1091B6: processa_lote (leak.c:4)
==12345==    by 0x109201: main (leak.c:15)
```



Interpretando a saída

Categoria	O que significa
definitely lost	memória perdida com certeza, sem ponteiro para ela
indirectly lost	perdida porque a struct que apontava para ela vazou
possibly lost	existe um ponteiro “estranho” apontando para ela
still reachable	não liberada, mas o programa ainda alcança ela

Meta

Depois de corrigir, a saída deve terminar assim:

```
All heap blocks were freed - no leaks are possible.
```



Notes

strace: rastreando chamadas de sistema

Notes

O que o strace mostra

Lá no Encontro 2 vimos que todo programa conversa com o kernel por meio de **chamadas de sistema** (*syscalls*).

- strace intercepta **cada** syscall feita pelo processo e a imprime
- Não precisa de -g, não precisa recompilar — funciona em **qualquer** binário
- Ótimo para responder: “que arquivo esse programa está tentando abrir?”, “por que ele está travado?”, “por que está lento?”



Notes

strace na prática

```
$ strace ./notas
execve("./notas", ["../notas"], 0x7ffd3c2a1e40 /* 64 vars */) = 0
brk(NULL)                               = 0x55f2a1b32000
openat(AT_FDCWD, "/etc/ld.so.cache", O_RDONLY|O_CLOEXEC) = 3
read(3, "\177ELF\2\1\1\3\0\0\0\0\0\0\0... ", 832) = 832
mmap(NULL, 8192, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_ANONYMOUS, -1, 0) = 0x7f5e...
write(1, "Media da turma: 8.00\n", 22) = 22
exit_group(0)                           = ?
```

- Cada linha: `syscall(argumentos) = retorno`
- `execve`, `brk`, `openat`, `mmap`... acontecem **antes** de qualquer linha do seu `main()` — é o *runtime* do C se preparando



Notes

Filtrando o que importa

```
1 $ strace -e trace=open,openat,read,write ./notas # so I/O
2 $ strace -c ./notas # resumo por syscall
3 $ strace -f ./programa # segue processos
   filhos (fork)
4 $ strace -o saida.txt ./programa # salva em arquivo
```

- `-e trace=` remove o ruído e mostra só o que interessa
- `-c` agrega **contagem e tempo** por tipo de syscall — ótimo ponto de partida para achar gargalo
- `-f` é essencial quando o programa usa `fork()/exec()`



Notes

Caso de uso: por que meu programa está lento?

```
$ strace -c ./copia_arquivo
% time    seconds  usecs/call   calls   syscall
-----
 78.42    0.041230      4    10240  read
 15.10    0.007942      4     1900  write
  4.33    0.002277    113      20  openat
  2.15    0.001130     56      20  close
-----
100.00    0.052579      12180 total
```

Diagnóstico

10240 chamadas de `read()` para copiar um arquivo? O buffer de leitura está **pequeno demais**. Aumentar o tamanho do buffer reduz drasticamente o número de syscalls (e o tempo total).



Notes

CMake: introdução

Notes

Por que CMake além do Make?

O problema

Seu Makefile funciona no seu Linux com GCC. E se alguém quiser compilar no macOS com Clang, ou gerar um projeto para o Visual Studio?

CMake resolve isso:

- Você descreve **o quê** compilar em `CMakeLists.txt` — não *como*
- O CMake **gera** o Makefile (ou Ninja, ou projeto do Visual Studio...) para a sua plataforma
- Padrão de fato na maioria dos projetos C/C++ modernos

Notes



```
1 cmake_minimum_required(VERSION 3.16)
2 project(turma C)
3
4 add_executable(turma main.c estatistica.c)
```

- `cmake_minimum_required` — versão mínima do CMake exigida
- `project(turma C)` — nome do projeto e linguagem usada
- `add_executable` — alvo executável e seus fontes (nada de `.o` manual!)



Build fora da árvore de código

```
1 $ mkdir build && cd build
2 $ cmake ..          # configura o projeto, gera o Makefile
3 $ make              # compila de verdade
4 $ ./turma
```

Por que uma pasta `build/`?

- Mantém o código-fonte **limpo** — nada de `.o` misturado com `.c`
- Para recomençar do zero: `rm -rf build/` — sem risco de apagar código
- `build/` entra direto no `.gitignore`



```
1 cmake_minimum_required(VERSION 3.16)
2 project(turma C)
3
4 set(CMAKE_C_STANDARD 11)
5 set(CMAKE_C_STANDARD_REQUIRED ON)
6
7 add_executable(turma main.c estatistica.c)
8 target_compile_options(turma PRIVATE -Wall -Wextra -g)
```

Compare com o Makefile da aula 02: mesmas flags, mesma ideia — só que o CMake calcula as dependências e a portabilidade para você.



Notes

Make vs. CMake

Aspecto	Make	CMake
Nível	regras manuais	descreve o alvo, CMake gera as regras
Portabilidade	Makefile fixo	gera Makefile/Ninja/VS conforme o SO
Uso típico	projetos pequenos	projetos grandes, multiplataforma

Make não desaparece — CMake usa (e gera) Makefiles por baixo dos panos.



Notes

Encerramento

Notes

Síntese: quatro ferramentas novas

Ferramenta	Pergunta que responde	Quando usar
Git	Como versionar e colaborar?	Sempre, desde o 1º commit
Valgrind	Onde a memória está vazando?	Uso de RAM cresce sem parar
strace	O que o programa pede ao kernel?	Depurar performance/permissões
CMake	Como organizar builds portáteis?	Projetos com vários arquivos

Notes

Junto com GCC, Make e GDB: as ferramentas do semestre inteiro.



Atividade prática de hoje

No laboratório, vocês vão:

- 1 Criar um repositório **Git** e praticar o fluxo `add/commit/branch/merge`
- 2 Encontrar e corrigir um **memory leak** com o **Valgrind**
- 3 Rastrear as chamadas de sistema de um programa com **strace**
- 4 Descrever e compilar um projeto com múltiplos arquivos usando **CMake**

Guia da atividade

Todas as instruções, código-fonte e critérios de avaliação estão no **guia de atividade prática** do Laboratório 02.



Notes

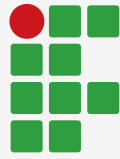
Bibliografia de apoio

- Manuais: `man git`, `man valgrind`, `man strace`, `man cmake`
- Pro Git (livro, PT-BR) – <https://git-scm.com/book/pt-br/v2>
- Valgrind User Manual – <https://valgrind.org/docs/manual/manual.html>
- CMake Documentation – <https://cmake.org/cmake/help/latest/>



Notes

That's all folks!



**INSTITUTO
FEDERAL**
Santa Catarina

Câmpus
Florianópolis



Notes

Notes
